Session 25 Teamwork — A Real Registration Flow

Context

Work in your team (six teams of four students). LIVE already built a real, working login: a Server Action (`loginAction`), a real httpOnly session cookie, a protected route group, and real 401 handling. Your job: build the matching registration flow, using the identical real pattern already demonstrated — proving that the pattern genuinely transfers, not inventing a new one.

Goal

A real `/register` route where a new user can create an account against the real backend, with a real duplicate-email error shown inline, and a real way to move between `/login` and `/register`.

What to build

- A real new Server Action that validates a name, email, and password are present (and that the password meets the real backend's own minimum length) before ever calling the backend — the same validate-first shape `loginAction` already uses.
- The real backend call against the registration endpoint, with the real duplicate-email response handled as an inline form error, never a thrown exception.
- Think carefully about where the new account ends up after a successful registration — should it behave exactly like a successful login, or is there a real reason to treat the two differently?
- A real form component, built the same way `LoginForm` was built live: start with the wiring, then add each field one at a time.
- A real `/register` page.
- Real links between `/login` and `/register` so a visitor on either page can reach the other.

Rules

Allowed: everything already shown LIVE this session (`"use server"`, `useActionState`, `apiFetch`, `requireSessionToken`), plus the real backend's own registration endpoint and DTO (already built in an earlier session — read it, don't guess its shape).

Not yet: no Redux Toolkit, no resolved user names anywhere, no Create Workspace, no Workspace Settings/member management — all of that is this session's own homework, not teamwork.

Test it

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
npx tsc --noEmit
```

Then, in a real browser: open `http://localhost:3001/register`, create a real new account, confirm you land somewhere sensible; try registering the SAME email again and confirm a real, inline error appears instead of a crash or a blank page.

Done when

- A real new account can be created end to end against the real backend.
- A real duplicate-email attempt shows an inline error, never a thrown exception or a generic crash page.
- `/login` and `/register` link to each other in both directions.
- `npx tsc --noEmit` exits clean.

Working time

About 27 minutes total (TW-01..08).